

Recent Research on Mitigating FaaS Cold Starts in Workflows (2020–2025)

Serverless **Function-as-a-Service (FaaS)** platforms suffer from the “cold start” problem, where invoking a function after an idle period incurs extra latency to initialize the runtime. This overhead is especially detrimental in **workflows** (function chains) since cold-start delay can compound across sequential steps ¹ ². Over the last five years, many solutions have been proposed to reduce cold-start latency – from workflow-aware preloading techniques to caching, scheduling, and machine-learning-based optimizations. Below is a selection of notable research (2020–2025) relevant to *reducing FaaS cold start latency*, especially in the context of multi-function workflows.

Surveys and Overviews

- **Cold Start Latency in Serverless Computing: A Systematic Review** (Golec *et al.*, ACM Computing Surveys 2024) – A comprehensive survey and taxonomy of cold start mitigation techniques ³ ⁴. It categorizes existing solutions (caching, application-level optimizations, AI/ML-based approaches, etc.) and analyzes their impact. This work (accepted to ACM CS) provides an updated overview of the state-of-the-art and outlines open challenges ⁵ ⁶. (DOI: 10.1145/3700875)

Workflow-Focused Cold Start Solutions

- **Using Application Knowledge to Reduce Cold Starts in FaaS Services** (Bermbach *et al.*, ACM SAC 2020) – Proposes a workflow-level “*choreography*” middleware that **pre-warms downstream functions** based on the known sequence of a function composition ⁷ ⁸. By sending *hints/triggers* ahead of actual invocations, their approach eliminates many cold starts during function chains. Experiments on AWS Lambda and IBM OpenWhisk show this method can remove ~40% (and up to 80%) of cold starts with minimal cost overhead ⁹. (DOI: 10.1145/3341105.3373909)
- **Xanadu: Mitigating Cascading Cold Starts in Serverless Function Chain Deployments** (Daw *et al.*, Middleware 2020) – Introduces *speculative just-in-time preloading* to tackle **cascading cold starts** in function workflows ¹⁰ ¹¹. Xanadu predicts which function will be invoked next (using most-likely execution paths) and spawns it *warm* in advance. This approach nearly **eliminates cold-start delays in function chains**, achieving up to 18× faster workflow execution vs. Knative and 10× vs. Apache OpenWhisk in their evaluations ¹² ¹³. (DOI: 10.1145/3423211.3425690)
- **Mitigating Cold Start Problem in Serverless Computing with Function Fusion** (Lee *et al.*, Sensors 2021) – Studied a **function fusion** strategy for serverless workflows. By **merging consecutive functions** into a single deployable unit, they remove the cold start of intermediate functions in a pipeline ¹⁴ ¹⁵. The authors present a workflow response-time model and an algorithm to decide optimal fusion without overly serializing parallel branches ¹⁶ ¹⁷. In tests on various workflows, their fusion approach reduced end-to-end latency to only 28–86% of the original (unfused) workflow time ¹⁸ ¹⁵. (DOI: 10.3390/s21248416)

- **“Scientific Workflow Scheduling in Cloud Considering Cold Start and Variable Pricing”** (Arslan *et al.*, arXiv preprint 2025) – Although focused on cloud VM provisioning, this work addresses cold-start delay in scheduling scientific workflows. It highlights that cold starts can add ~20% overhead to workflows ¹⁹. The proposed two-phase scheduler (with caching of warm resources and dynamic VM allocation) explicitly accounts for task dependencies and cold start costs ¹⁹ ²⁰. Experimental results show ~20% improvement in workflow makespan over prior cold-start-unaware methods ²¹. (*arXiv:2504.21536*)

General Techniques to Reduce FaaS Cold Starts

- **FaaSCache: Keeping Serverless Computing Alive with Greedy-Dual Caching** (Fuerst & Sharma, ASPLOS 2021) – A caching-inspired “keep-alive” policy for FaaS platforms ²². Instead of letting function instances terminate immediately, FaaSCache uses a Greedy-Dual algorithm (adapted from web caching) to decide which idle functions to keep alive in memory. Implemented on Apache OpenWhisk, it cut cold-start frequency and overhead by 3× or more compared to default policies ²³. This caching approach improves warm reuse while still bounding resource use ²⁴.
- **HotC: Adaptive Container Runtime Reuse** (Suo *et al.*, IEEE Cluster 2021) – An efficient **container reuse** framework to tackle cold starts. It keeps a pool of “hot” runtime containers and intelligently decides when to reuse them. HotC uses workload prediction (exponential smoothing + Markov chain) to maintain just enough warm containers ²⁵. By reusing containers across function invocations, it mitigates cold start delays without excessive resource waste. (*DOI: 10.1109/Cluster48925.2021.00018*)
- **Agile Cold Starts via Pause Containers** (Mohan *et al.*, HotCloud 2019) – (Included for context) Prototyped on OpenWhisk, this solution pre-creates and caches network stack components (using “pause” containers) so that new function containers can attach to an already-initialized network interface ²⁶ ²⁷. This reduced container startup latency under high concurrency by an order of magnitude (e.g. ~80% lower startup time at 100 concurrent cold launches) ²⁸ ²⁹. (HotCloud’19 workshop)*
- **Incendio: Priority-Based Scheduling for Alleviating Cold Start** (Cai *et al.*, IEEE T. Computers 2024) – Proposes a serverless framework “Incendio” that uses **priority-aware scheduling** to minimize cold-start impact. Frequently-invoked functions are given priority in scheduling/placement, reducing cold start occurrences in latency-critical requests ³⁰. This approach schedules functions based on invocation patterns, improving overall response times in serverless workloads. (*IEEE Trans. Computers, vol. 73, no. 7, 2024*)
- **RL-Based Adaptive Prewarming (IoT-J 2022)** – Vahidinia *et al.* (IEEE Internet of Things Journal, 2022) present a two-layer solution with reinforcement learning to time container pre-warming. An RL agent learns invocation patterns to decide when to keep containers alive, and an LSTM predicts upcoming function invocations ³¹ ³². Tested on OpenWhisk, their method cut down wasted warm instances (saving ~12% memory) while increasing the fraction of requests served on pre-warmed containers by ~22% ³³.
- **Holistic Cold-Start Management with Time-Series Prediction** (Nguyen *et al.*, FGCS 2024) – Introduces a **two-pronged policy** that coordinates cold start handling across function, application, and infrastructure levels ³⁴ ³⁵. A Temporal Convolutional Network (TCN) predicts function arrival

rates 5–15 minutes ahead, allowing proactive instance pre-spawning. This AI-driven policy operates with feedback loops to higher-level schedulers. The evaluation on real traces showed reliable prediction accuracy and improved SLA compliance ³⁵ . (DOI: 10.1016/j.future.2023.12.011)

- **Container Resource Reuse via Namespace Carving** (Li *et al.*, *Electronics* 2023) – A novel **container-space reuse** mechanism to shorten cold-start times ³⁶ ³⁷ . It classifies function containers by utilization and “steals” unused memory/CPU from low-usage containers using Linux namespace and cgroups tricks ³⁷ ³⁸ . The freed resources form *pre-warmed mini-containers* for new functions. In tests, this method reduced cold start times to <20 ms and mitigated 90% of cold starts **without** any always-on warming pool ³⁷ . (DOI: 10.3390/electronics12112515)
- **Transformer-Based Cold Start Mitigation** (Mbala Mouen *et al.*, arXiv preprint 2025) – Explores deep learning for cold-start prediction. This work uses a **Transformer model** to forecast function invocation patterns and proactively allocate resources ³⁹ . Experiments (with Azure Functions traces and OpenWhisk simulation) demonstrated significant latency reductions – up to **79% faster cold starts** versus conventional heuristics ³⁹ . This showcases the potential of modern AI (time-series forecasting with Transformers) in optimizing serverless performance.

Each of the above papers offers insights or techniques to reduce FaaS cold start delays. For a thesis on improving cold starts in workflows, the **workflow-aware methods** (e.g. Bermbach 2020, Daw 2020, Lee 2021) will be most directly relevant, while the general techniques (caching, scheduling, reuse, ML prediction) provide additional ideas that can be applied or extended within workflow contexts. All references listed are from roughly the last five years, aligning with current trends in serverless research. Please refer to the DOI or links provided for each paper to find the full publications.

¹ ² ¹⁰ ¹¹ ¹⁴ ¹⁵ ¹⁶ ¹⁷ ¹⁸ Mitigating Cold Start Problem in Serverless Computing with Function Fusion

<https://www.mdpi.com/1424-8220/21/24/8416>

³ ⁴ ⁵ ⁶ [2310.08437] Cold Start Latency in Serverless Computing: A Systematic Review, Taxonomy, and Future Directions

<https://arxiv.org/abs/2310.08437>

⁷ ⁸ Using Application Knowledge to Reduce Cold Starts in FaaS Services

<https://dbermbach.github.io/publications/2020-sac-cc.pdf>

⁹ (PDF) Using application knowledge to reduce cold starts in FaaS services

https://www.researchgate.net/publication/340268720_Using_application_knowledge_to_reduce_cold_starts_in_FaaS_services

¹² ¹³ (PDF) Xanadu: Mitigating cascading cold starts in serverless function chain deployments

[https://www.researchgate.net/publication/](https://www.researchgate.net/publication/347698258_Xanadu_Mitigating_cascading_cold_starts_in_serverless_function_chain_deployments)

347698258_Xanadu_Mitigating_cascading_cold_starts_in_serverless_function_chain_deployments

¹⁹ ²⁰ ²¹ Scientific Workflow Scheduling in Cloud Considering Cold Start and Variable Pricing Model

<https://arxiv.org/html/2504.21536v1>

²² ²³ ²⁴ FaasCache: Keeping Serverless Computing Alive with Greedy-Dual Caching

<https://afuerst.github.io/assets/FaasCache.pdf>

25 Tackling Cold Start of Serverless Applications by Efficient and Adaptive Container Runtime Reusing | Request PDF

[https://www.researchgate.net/publication/](https://www.researchgate.net/publication/355227058_Tackling_Cold_Start_of_Serverless_Applications_by_Efficient_and_Adaptive_Container_Runtime_Reusing)

355227058_Tackling_Cold_Start_of_Serverless_Applications_by_Efficient_and_Adaptive_Container_Runtime_Reusing

26 27 28 29 [usenix.org](https://www.usenix.org)

<https://www.usenix.org/system/files/hotcloud19-paper-mohan.pdf>

30 Priority-Based Scheduling for Alleviating Cold Start in Serverless ...

[https://www.researchgate.net/publication/379674886_Incendio_Priority-](https://www.researchgate.net/publication/379674886_Incendio_Priority-based_Scheduling_for_Alleviating_Cold_Start_in_Serverless_Computing)

based_Scheduling_for_Alleviating_Cold_Start_in_Serverless_Computing

31 32 33 Mitigating Cold Start Problem in Serverless Computing: A Reinforcement Learning Approach

[http://faculty.washington.edu/wlloyd/courses/tcss591/papers/](http://faculty.washington.edu/wlloyd/courses/tcss591/papers/Mitigating_Cold_Start_Problem_in_Serverless_Computing_A_Reinforcement_Learning_Approach.pdf)

Mitigating_Cold_Start_Problem_in_Serverless_Computing_A_Reinforcement_Learning_Approach.pdf

34 35 Holistic cold-start management in serverless computing cloud with deep learning for time series | Request PDF

[https://www.researchgate.net/publication/376499318_Holistic_cold-](https://www.researchgate.net/publication/376499318_Holistic_cold-start_management_in_serverless_computing_cloud_with_deep_learning_for_time_series)

start_management_in_serverless_computing_cloud_with_deep_learning_for_time_series

36 37 38 A Fast Cold-Start Solution: Container Space Reuse Based on Resource Isolation

<https://www.mdpi.com/2079-9292/12/11/2515>

39 Transformer-Based Model for Cold Start Mitigation in FaaS Architecture

<https://arxiv.org/html/2504.11338v1>